Siemens
Industry
Online
Support

EXAMPLE

Parallel record access to MultiFieldbus devices

MultiFieldbus

SIEMENS

Legal Notices

Use of application examples
In the application examples, the solution of automation tasks in the interaction of several components in the form of text, graphics and/or software modules is presented as an example. The application examples are a free service provided by Siemens AG and/or a subsidiary of Siemens AG ("Siemens"). They are non-binding and do not claim to be complete and functional in terms of configuration and equipment. The application examples do not represent customersspecific solutions, but only offer assistance with typical tasks. **The application examples are not subject to the standard testing and quality checks of a paid product and may contain functional and performance defects and other bugs and security vulnerabilities. You are responsible for the proper and safe operation of the products in accordance with all applicable regulations, including reviewing and adapting the use case for your system, and ensuring that only trained personnel use it in such a way that that property damage or injury to persons is avoided. You are solely responsible for any productive use.**

Siemens grants you the non-exclusive, non-sublicensable and non-transferable right to use the application examples by professionally trained personnel. Any changes to the application examples are at your responsibility. Passing on to third parties or reproducing the application examples or excerpts from them is only permitted in combination with your own products. Any further use of the application examples is expressly prohibited and no further rights are granted. You may not use the application examples in any other way, in particular any direct or indirect training or enhancement of AI models is expressly prohibited.

Disclaimer
Siemens excludes its liability, regardless of the legal grounds, in particular for the usability, availability, completeness and absence of defects of the application examples, as well as the associated information, project planning and performance data and any damage caused thereby. This does not apply to the extent that Siemens is liable, e.g. under the Product Liability Act, in cases of intent, gross negligence, culpable injury to life, limb or health, non-compliance with a warranty assumed, fraudulent concealment of a defect or culpable breach of essential contractual obligations. However, the claim for damages for the breach of essential contractual obligations is limited to the foreseeable damage typical for the contract, unless there is intent or gross negligence or liability is incurred due to injury to life, limb or health. A change in the burden of proof to your detriment is not associated with the above provisions. You indemnify Siemens against claims by third parties existing or arising in this context, unless Siemens is legally liable.

By using the application examples, you acknowledge that Siemens cannot be held liable for any damages beyond the liability policy described.

Further information
Siemens reserves the right to make changes to the Application Examples at any time without notice and to terminate the license to you at any time. In the event of discrepancies between the suggestions in the application examples and other Siemens publications, such as catalogs, the content of the other documentation takes precedence.

In addition, the Siemens Terms of Use (https://www.siemens.com/global/en/general/terms-of-use.html).

Cybersecurity Considerations
Siemens offers products and solutions with industrial cybersecurity functions that support the secure operation of plants, systems, machines and networks.

In order to secure plants, systems, machines and networks against cyber threats, it is necessary to implement (and continuously maintain) a holistic industrial cybersecurity concept that corresponds to the current state of the art. Siemens products and solutions form part of such a concept.

Customers are responsible for preventing unauthorized access to their facilities, systems, machines and networks. These systems, machines and components should only be connected to the company network or the Internet if and to the extent necessary and only if appropriate protective measures (e.g. firewalls and/or network segmentation) have been taken.

Further information on possible protective measures in the field of industrial cybersecurity can be found at www.siemens.com/cybersecurity-industry.

Siemens products and solutions are constantly being developed to make them even safer. Siemens strongly recommends that product updates be applied as soon as they are available and that only the latest product versions be used. Using outdated or deprecated versions can increase the risk of cyber threats.

To stay informed about product updates, subscribe to the Siemens Industrial Cybersecurity RSS Feed at https://www.siemens.com/cert.

Table of contents

1. Introduction

1.1. Overview

In addition to their primary function of producing goods, machines and systems also generate a wealth of valuable information. This information may include details such as the types of automation components installed, their duration of use, current hardware and firmware statuses, as well as any pending diagnostics.

The machine status of a system can also be read out. By analysing temperature profiles, vibration behaviour or energy consumption, it is possible to evaluate whether components are functioning within the expected range or whether there is a potential problem. Such evaluations can be used to intervene preventively to avoid major damage and failures.

It is important to note that this information is not part of the actual automation task and should not be processed in the controller. This detour leads to unnecessary load on the PLC and, depending on the scope, would also require stronger hardware.

This application example shows a way to obtain additional information from the field and peripheral level without having to adapt or expand an automation solution based on PROFINET. Neither the control program nor the basic hardware structure has to be changed.

In the best case, it is sufficient to replace the interface module of the SIMATIC peripheral station used.

NOTES

- For this application example, knowledge of Modbus communication and high-level language programming is required. The SIMATIC system can provide the data in simple steps. However, how the data is read, processed and evaluated is up to the user.
- This use case does not take into account aspects of IT security.

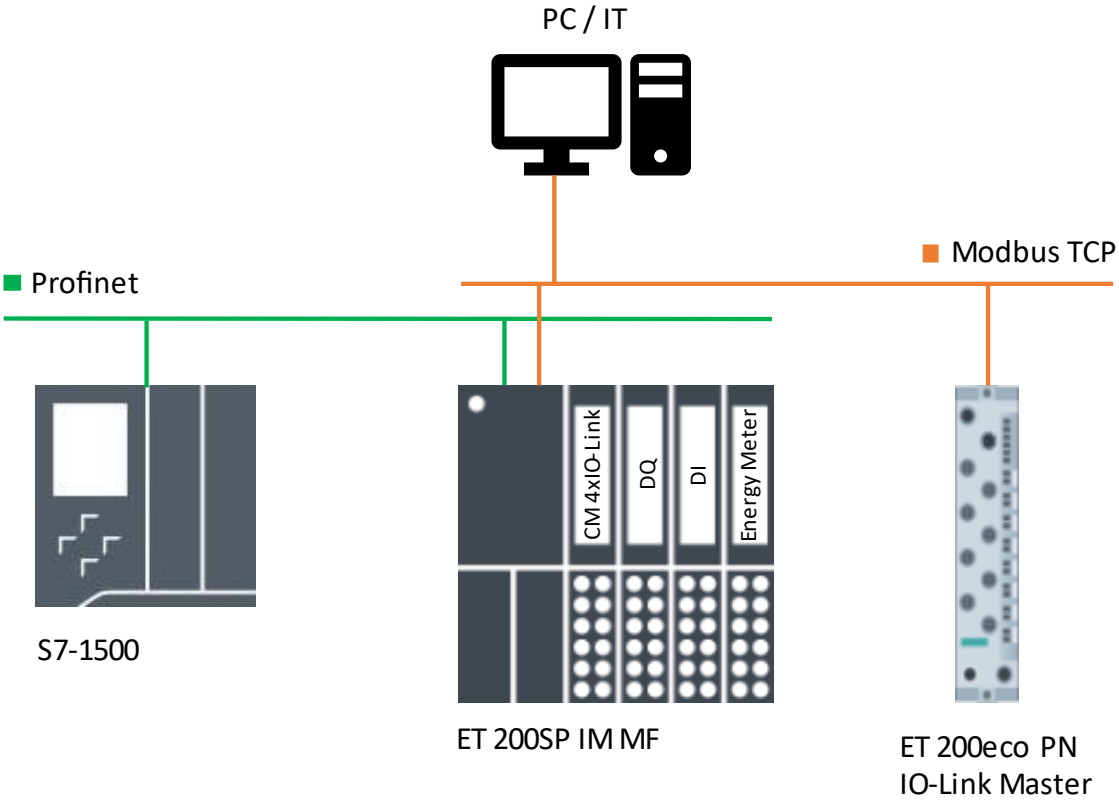
This application example provides an introduction and examples of how the data connection is established in different environments.

1.2. Functionality

For the application example, we use a very simplified setup with a PLC 1513-1PN, an ET 200SP assigned to it via PROFINET and an ET 200eco PN connected only to IT or to our PC. In order to get to the information mentioned, we use a Modbus/TCP communication parallel to the active PROFINET channel to access all data of the ET 200SP.

Make sure that it is an interface module with MultiFieldbus communication, specifically IM 155-6MF HF (or its successor). In addition to PROFINET IO, this IM supports Modbus/TCP and EtherNet/IP communication protocols.

This adaptation to an existing hardware solution is necessary in order to be able to use a Modbus/TCP channel in parallel. This channel gives you access to all data records of all modules of a SIMATIC ET 200 station. The ET 200eco PN, on the other hand, offers only one communication channel that can be configured as a Modbus/TCP channel and is therefore ready for communication with IT



In principle, this Modbus/TCP channel can be used by any programmable Modbus/TCP client. This can be realized by another control system or via high-level languages (C, Java, Python,...) on an IPC or microcontroller. In the application example, a simple Windows 10 PC is used.

1.3. Components Used

This application example was created with these hardware and software components:

Component	Number	Article	NOTE
TIA Portal V18			Or higher
MultiFieldbus Configuration Tool			Link to download
Port Configuration Tool			Link to download
Visual Studio Code			
PLC 1513-1PN	1	6ES7513-1AL02-0AB0	
IN 155-6MF HF	1	6ES7155-6MU00-0CN0	Firmware V5.2 or higher
CM 8xIO-Link + DI 4x24VDC	1	6ES7148-6JG00-0BB0	
DI 8x24VDC HF	1	6ES7131-6BF00-0CA0	
DQ 8x24VDC/0.5A HF	1	6ES7132-6BF00-0CA0	
CM 4xIO-Link ST	1	6ES7137-6BD00-0BA0	
AI Energy Meter RC HF	1	6ES7134-6PA21-0CU0	

You can use the listed components via the Siemens Industry Mall cover.

This use case consists of the following components:

Component	Filename	NOTE
Documentation	109987554_PN_MB_Parallel_Access_DOKU_V10_de.pdf	
TIA Example Project	parallel_dataset_access_to_MultiFieldbus_devices - TIA Project.zip	
MFCT Project	MFCT_Project.mfp1	
Python Program	main.py	Application file
	mfMb.py	Class file: MfMbCtrl, Et200SpMf and IOLinkMaster
	const.py	Application and Python Class Constants

With the exception of this documentation, these are external downloads.

External Downloads

This application is part of the MultiFieldbus Open Source Projects.

The TIA project, the MFCT project and the source code of this application are available as external downloads: https://github.com/MultiFieldbus/parallel-dataset-access-to-MultiFieldbus-devices.git.

NOTE The prerequisite for access to the projects and the source code is the login to a valid user account at Github.

If you are not logged in, you will receive a "404-File not found" error message.

2. Engineering

2.1. Hardware construction

The central control system is implemented by a PLC 1513-1PN. As a peripheral station, an ET 200SP with a MultiFieldbus-capable interface module is assigned to the S7-1500 as a PROFINET device at the X1 interface. In addition to a digital input card, the ET 200SP has a digital output card, an IO-Link master and an analog input card Energy Meter RC HF.

On the IO-Link master, the control cabinet temperature is read in via a SIRIUS 3RS2 using PT100.

With the energy meter, we read the average active power L1, the energy meter of the machine, the current voltage and current as well as their minimum/maximum with time stamp. For this purpose, the energy meter has been expanded to include a current-voltage converter.

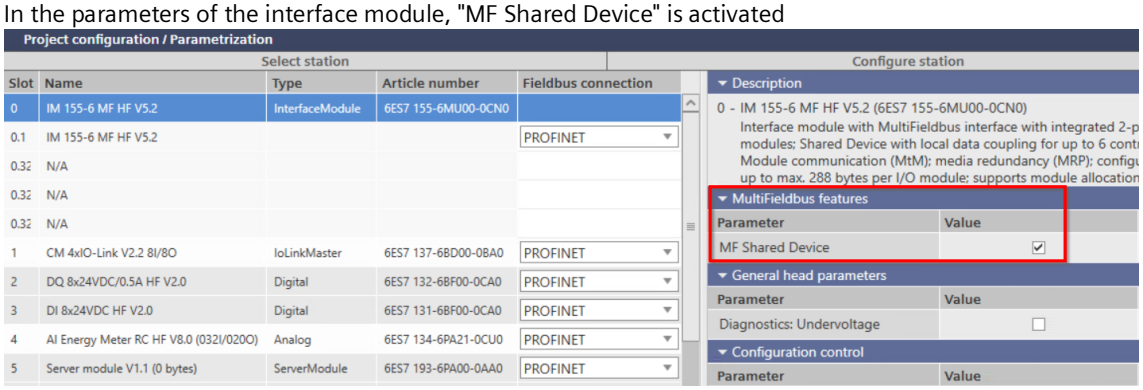
2.2. Configuration

The configuration in the TIA Portal is limited to the hardware configuration and the call to the PCT to put the IO-Link Master into operation. The control program remains empty with the OB1. The focus of this application example is on parallel data access and not on the control task.

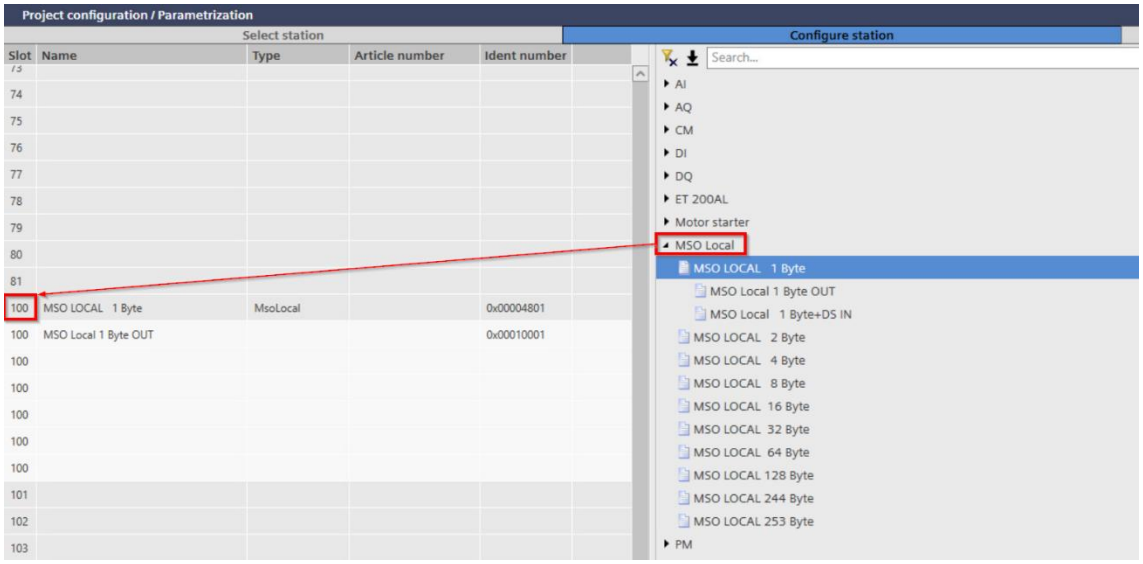
After loading the project onto the control system and correctly assigning the PN name to the ET 200SP, the system is in a comparable state as after commissioning and would be ready to be switched on. For the time being, however, we leave the PLC in the STOP state.

To open the Modbus/TCP channel in the interface module of the ET 200SP, we use the MFCT (reference: see chap. 1.3) and project the station again. However, there are three differences between the TIA Portal project planning and the MFCT project.

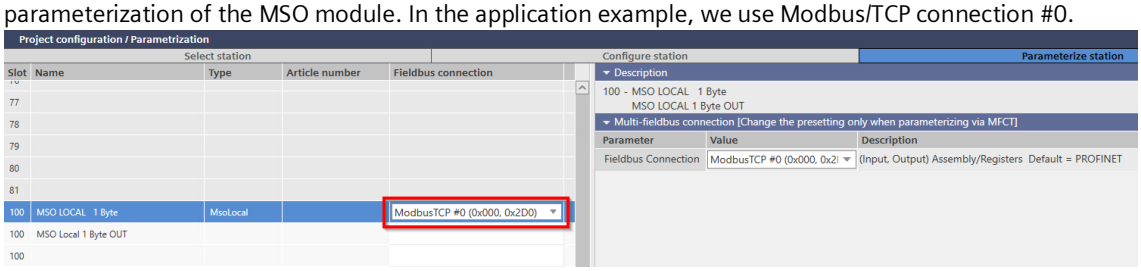
1. In the parameters of the interface module, "MF Shared Device" is activated



2. The actual modules are supplemented by a virtual "MSO Local" module in project planning. The "MSO Local" module can be inserted from slot 100.



3. Since the Shared Device function is active, a Modbus/TCP connection can be selected as a fieldbus connection in the parameterization of the MSO module. In the application example, we use Modbus/TCP connection #0.



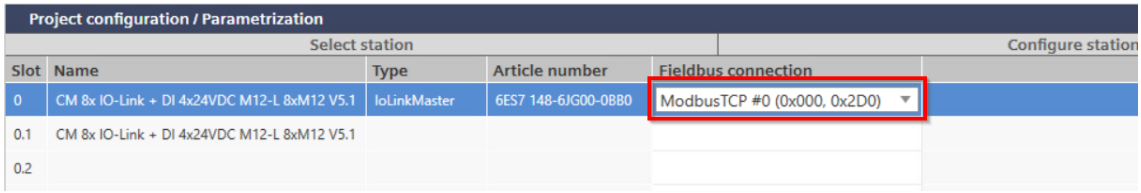
The virtual MSO module is only used to activate the Modbus/TCP channel in the IM155-6MF HF and has no other function beyond that.

The MFCT configuration is transferred to the corresponding device. However, the PLC must be disconnected from the device for this. The device refuses any communication as long as it is in a communication relationship (AR – "application relation") with a PROFINET IO controller. This is active even if the PLC is in STOP and has a valid hardware configuration.

After loading the MFCT configuration, the PLC is reconnected to the device and can be set to RUN. The additional MSO module is irrelevant for the PLC because it is not present in the TIA configuration. The Modbus/TCP channel is now available due to MFCT configuration.

Since the ET 200eco PN has only one network interface and is only connected to IT in our use case, it is necessary to switch the communication channel from PROFINET (default setting) to Modbus/TCP.

4. Select ModbusTCP #0 as the fieldbus connection for the ET 200eco PN module.



Modbus/TCP communication with MultiFieldbus and Raspberry Pi

https://support.industry.siemens.com/cs/document/109808268

Details on how to convert the communication protocol of an ET 200eco PN from PROFINET (default setting) to Modbus/TCP can be found in the chapter "Project planning of the ET 200eco PN module".

2.3. Data recording

The cyclic IO data of the device is exchanged with the PLC in the IM155-6MF HF as usual via PROFINET. The Modbus/TCP channel does not have direct access to the cyclic data, because except for the MSO module, none of the IO modules is assigned to a Modbus channel. The hold register area of the inputs and outputs is therefore empty. However, the virtual MSO module provides access to the dataset interface and thus to all datasets of all modules, including the interface and server modules.

If you also want to read the cyclic data of a module via Modbus/TCP, the corresponding data record must be read:

- Process Input Image: Record Index 8028
- Process Output Image: Record Index 8029

Take into account that this is an acyclic record access and that the update rate is much lower than a PROFINET IO access. The data in the IO controller and the data read out via Modbus/TCP are therefore not consistent in time.

For record access via Modbus/TCP to a MultiFieldbus device, a three-step access procedure is used, which can be summarized as follows:

1. Writing the order in Register A

2. Reading the Order Status from Register B
3. Reading the requested record from Register C

To write records via Modbus/TCP to a MultiFieldbus device, the 3-step access method is also used, but with a different order of register access:

1. Writing the data to Register C
2. Writing the order in Register A
3. Reading the Order Status from Register B

SIMATIC MultiFieldbus Function Manual

https://support.industry.siemens.com/cs/document/109773209

Details on the use of the dataset interface of MultiFieldbus devices can be found in the chapter "Modbus/TCP" under the item "Record Interface".

A, B and C depend on which Modbus connection you have selected in the MFCT for the MSO module.

For the Modbus #0 connection, the following register values apply to record access:

Register structure	Start address	Length
Record Request (A)	0xD000 (53248)	6 Registers
Record Response (B)	0xD040 (53312)	3 Registers
Record Content (C)	0xD0FA (53376)	123 Registers

The remaining data record registers for connections #1, #2, ... are offset by 256 registers each. For example, the record request for connection #1 is 0xD100 (53504).

In order to gain access to the data sets, the described procedure must be implemented and called using a Modbus/TCP client instance in a corresponding application.

Communication with the ET 200eco PN takes place via data record access in the same way as with an IM155-6MF HF. The only difference is that the ET 200eco PN has only one network interface and therefore only one protocol can be used at a time.

To retrieve IO-Link data, an IOL-Call is used, which is implemented with the method for reading/writing records in MultiFieldbus Devices.

To read process data from an IO-Link device with an IOL-Call, the following indexes are used:

- Process Data Input Index 40
- Process Data Output Index 41

3. IT Integration

3.1. Interface description

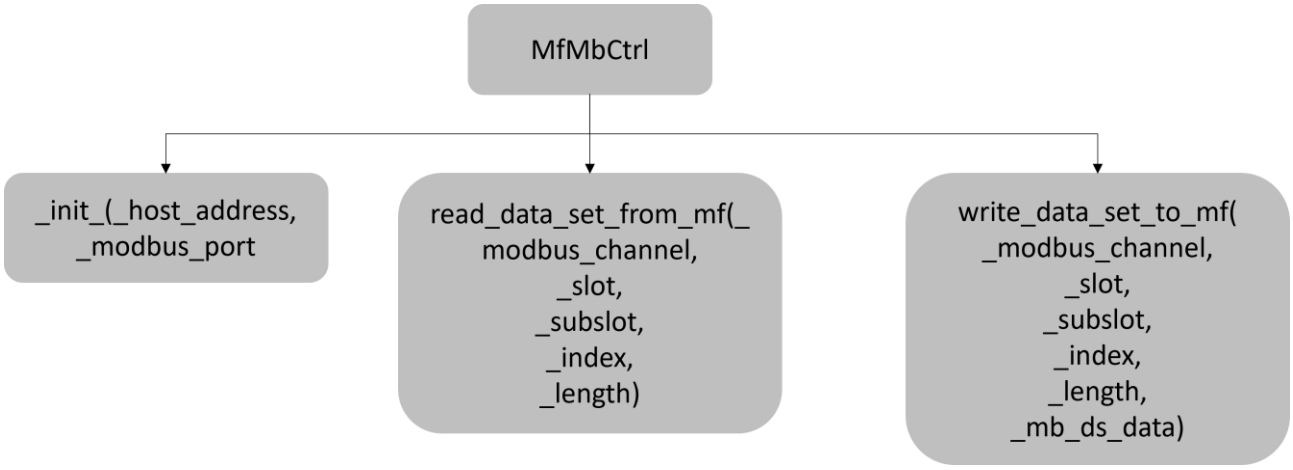
The Modbus communication to the ET200SP MF Station and also to the IO-Link Master was implemented as an application using the programming language "Python". The file `mfMb.py` contains three classes: `MfMbCtrl`, `Et200SpMf` and `IOLinkMaster`. The constants used by these classes are stored in the `const.py` declares.

3.1.1. The Class `MfMbCtrl`

The `MfMbCtrl` Class enables Modbus/TCP communication with MultiFieldbus devices. It provides methods for connecting to the devices and reading and writing records to the devices.

`MfMbCtrl` can serve as a basis for communication with MultiFieldbus-capable devices.

Main functions of the `MfMbCtrl`-Library



Initialization

The constructor of the class "`__Init__`" requires the parameters "`_host_address`" (IP address) and "`_modbus_port`" to establish a connection with the MF-capable peripherals.

Reading Records

The method "`read_data_set_from_mf`" reads records from an MF-capable device. Here, the 3-step method for data access mentioned above is implemented.

It requires the parameters "`_modbus_channel`", "`_slot`", "`_sub_slot`", "`_index`" and "`_length`" to define the specific record. The method returns a list of the read record.

Writing Records

The method "`write_data_set_to_mf`" is used to write a record to the MF-capable device. Here, the 3-step method of writing records mentioned above is implemented.

It uses parameters similar to the method used to read data. However, in addition to the parameters for reading, this method also requires the parameter "`_mb_ds_data`", which should be a list of integers that represent the data to be written. The method returns the confirmation status, which indicates the completion of the write operation.

Example usage

This example shows the reading of the I&M0 Profinet record (45040, 16#AFF0) and the writing of the date to the I&M2 Profinet record (45042, 16#AFF2). (Use of the `MfMbCtrl` class for creating a client and reading/writing records from/to the MF device)

```
# Client initialisieren
modbus_client = MfMbCtrl(_host_address="192.168.1.10", _modbus_port=502)

# I&M0 Profinet Record vom Gerät lesen
data = modbus_client.read_data_set_from_mf(_modbus_channel=0, _slot=0,
_sub_slot=1, _index=45040, _length=60)

# Die relevanten Daten abholen
article_number = data[4:14]

# Das Installationsdatum (Type: Array of Hex) des Moduls in den I&M2 Profinet
Record schreiben
schreib_status = modbus_client.write_data_set_to_mf(_modbus_channel=1, _slot=2,
_sub_slot=3, _index=4, _length=50, _mb_ds_data=im_date)

# Den Schreibstatus ausgeben
print("Schreibstatus:", schreib_status)
```

3.1.2. The Class `Et200SpMf` (inherits from `MfMbCtrl`)

The Class `Et200SpMf` provides an interface to interact with Siemens ET200SP MF hardware. It enables the retrieval of identification and maintenance data (I&M) for asset management, as well as the collection of diagnostic data for preventive/predictive maintenance. Additionally, the class enables input and output (IO) data to be read, supporting real-time monitoring of equipment and processes to optimize operational efficiency[1]. This class has the following properties:

- Initialization: The class allows the initialization of the communication (by the constructor `__Init__`) with the ET200SP MF hardware via the host IP address, Modbus port number, and available IO-Link master ports. The available IO-Link Master Ports are an array of ports. For example, for an IO-Link master with 4 ports, the array can be of the form ["Port 1", "Port 2", "Port 3", "Port 4"]. This is needed for handling, to store data in a structured form and to avoid errors. In the file `const.py` two possible arrays (for IO-Link master with 4/8 ports) are available, which can be used to initialize an instance of the ET200SpMf in the user application, depending on the layout of the rack.
- The Constructor `__Init__` creates these instance variables:
  - `self.device_im_data` - In the form of a DataFrame to store the device I&M data of the individual modules of the rack
  - `self.diagnose_data` – in the form of a DataFrame to store diagnostic data
  - `self.iolink_devices` – in the form of a DataFrame to store information about the IO-Link devices connected to the master
- Use these instance variables to access the information you need. The information contained in these instance variables must be updated before use by calling the appropriate methods, which are explained below.
- Retrieving Identification and Maintenance (I&M) Data: The class provides a method "`get_device_im_data()`, which retrieves the I&M data for each module connected to the hardware. This includes information such as item number, serial number, hardware version, and firmware version.
- PROFINET diagnostics of channels with errors: The class provides a method "`get_diagnose()`", which retrieves diagnostic data for channels with errors in the hardware. It populates the instance variable `self.diagnose_data` with information such as slot, submodule, channel type, error code and error message.
- Reading IO data: Use the "`read_io_data()`"to read IO data from the sensors. This includes retrieving temperature information and the status of digital inputs/outputs.

[1] PROFINET data sets are used to read the I&M data, diagnostic data and IO data. The detailed description of the PROFINET data sets can be found under \8\.

- Reading Energy Data from the Energy Meter: The class provides a method "`read_energy_meter_data()`", which reads energy data from the Energy Meter. The data read for this use case is current, voltage, active power, energy meter, maximum current (with timestamp) and maximum voltage (with timestamp)[2].
- Configuring IO-Link Devices: The "`liolink_device()`" enables the configuration of IO-Link devices via Modbus communication. It supports tasks such as reparameterization, diagnostics and execution of port functions. It returns the data read by the IO-Link device or written to the device in the form of an array of hexadecimal numbers.
- Automatic Detection of IO-Link Devices: The "`find_iolink_devices()`" facilitates the detection of IO-Link devices that are connected to the IO-Link master (CM 4x IO-Link). It retrieves information such as manufacturer name and product name. In the end, the instance variable `self.iolink_devices` updated with the information.
- Auxiliary method: Removing leading zeros. The auxiliary method "`remove_trailing_zeros()`" removes leading zeros from an array (0x000A => 0xA) to improve data processing and readability.

`Et200SpMf` inherits from the class `MfMbCtrl`, which means that all methods in `MfMbCtrl` also in the class `Et200SpMf` can be used.

NOTE Note that the application will not work if a module is not plugged in. This application error can be avoided by reading the PROFINET diagnostics, which indicates the number of available modules connected to the peripherals.

Example usage

This example explains how to use the class `Et200SpMf`. These include

- How to create a client,
- How the I&M data, diagnostic data, IO data and some data are read by the energy meter,
- How to use the "`liolink_device()`"[3] is used to read parameters of IO-Link devices, and
- How to automatically connect all IO-Link devices connected to the master via "`find_iolink_devices()`".

- Example of creating an Et200SpMf client and reading I&M data from the MF device:

Code:

```
# Initialisierung eines et200spMF-Clients
et200sp = Et200SpMf(_host_address="192.168.1.10", _modbus_port=502,
_iolink_master_ports=["Port 1","Port 2","Port 3", "Port 4"])

# Abrufen und Anzeigen von I&M-Daten der Peripherie ET200SP und der daran
angeschlossenen Module
et200sp.get_device_im_data()
print(et200sp.device_im_data)
```

Debugging:

	IM155-6MF HF	CM 4xIO-Link ST	DQ 8x24VDC/0.5A HF	DI 8x24VDC HF	AI Energy Meter ST
I&M - Information					
Article Number	6ES7 155-6MU00-0CN0	6ES7 137-6BD00-0BA0	6ES7 132-6BF00-0CA0	6ES7 131-6BF00-0CA0	6ES7 134-6PA21-0CU0
Serial Number	S C-P3MR22772022	S C-D2VA99582013	S C-JDNH47282017	S C-JDLV90782017	S C-P3QD65812022
HW Revision	3	1	5	6	2
FW Version	V 5.2.2	V 2.2.2	V 2.0.2	V 2.0.1	V 8.0.0

[2] AI Energy Meter unterstützt das Ablesen von Max- und Min-Werten mit Zeitstempel. Voraussetzung dafür ist, dass das MF-Kopfmodul mit der CPU synchronisiert wird. Dies wird extern durchgeführt. Siehe hierzu das SiePortal Anwendungsbeispiel \6\.
Diese Funktionalität ist bereits in das TIA-Projekt integriert, das in dieser Anwendung bereitgestellt wird.
[3] Die Methode "`liolink_device()`" ist eine ähnliche Implementierung des `LIOLink_Device` Funktionsblocks der LIOLink-Library. Siehe \7\ für den Download der Bibliothek für IO-Link (LIOLink) für Simatic Steuerungen.

- Example of reading the diagnostic data and the input/output data from an MF device:

Code:

```
# Abrufen und Anzeigen von Diagnosedaten
et200sp.get_diagnose()
print(et200sp.diagnose_data)

# Lesen und Anzeigen von IO-Daten
temperature, digital_outputs, digital_inputs = et200sp.read_io_data()
print("Temperatur:", temperature)
print("Digitale Ausgänge:", digital_outputs)
print("Digitale Eingänge:", digital_inputs)
```

Debugging:

```
   Slot        Submodule Channel type Channel number Error code Error message
0    1  CM 4xIO-Link ST     Reserved              2        0x6    Drahtbruch
1    3    DI 8x24VDC HF       Output              0        0x6    Drahtbruch
2    3    DI 8x24VDC HF       Output              1        0x6    Drahtbruch
Temperature:  26
Digital Outputs:  ['FALSE', 'TRUE', 'FALSE', 'FALSE', 'FALSE', 'FALSE', 'FALSE', 'FALSE']
Digital Inputs:  ['FALSE', 'TRUE', 'FALSE', 'FALSE', 'FALSE', 'FALSE', 'FALSE', 'FALSE']
```

- Example of reading energy measurement data:

Code:

```
# Lesen und Anzeigen von Energiemessdaten
current, voltage, power, energy_counter, max_current, max_current_timestamp,
max_voltage, max_voltage_timestamp = et200sp.read_energy_meter_data(triggerEner-
gyCounter=True)
print("Strom:", current)
print("Spannung:", voltage)
print("Leistung:", power)
print("Energiezähler:", energy_counter)
print("Maximaler Strom:", max_current)
print("Zeitpunkt des maximalen Stroms:", max_current_timestamp)
print("Maximale Spannung:", max_voltage)
print("Zeitpunkt der maximalen Spannung:", max_voltage_timestamp)
```

Debugging:

```
Strom: 4.00972900390625
Spannung: 242.7554168701172
Leistung: 55966.81640625
Energiezähler: -2.3324852734804153
Maximaler Strom: 6.919444580078125
Zeitpunkt des maximalen Stroms: 2023-08-28 17:18:58
Maximale Spannung: 246.2565460205078
Zeitpunkt der maximalen Spannung: 2023-08-26 14:32:22
```


- Example of an IOL-Call with the aim of reading some parameters from an IO-Link device connected to the master, and at the same time finding all IO-Link devices connected to a master:

Code:

```
# Auslesen von Direct Parameters eines IO-Link Devices
iolink_data = et200sp.liolink_device(_modbus_channel=0, _slot=1, _sub_slot=1,
_port=4, _cap=227, _readWrite=0,_index=0, _subindex=0, _length=20, _writ-
ingRecord=[])
print("Direct Parameters:", iolink_data)

# IO-Link Devices suchen und anzeigen
et200sp.find_iolink_devices(_modbus_channel=0, _slot=1, _sub_slot=1)
print(et200sp.iolink_devices)
```

Debugging:

```
Direct Parameters:  ['0x804', '0xfe4a', '0x0', '0x0', '0x32', '0x3211', '0x1183', '0x1000', '0x2a09', '0x730']
              Port 1 Port 2 Port 3          Port 4
Identification
Product Name     None   None   None  3RS2800-*BA40
Vendor Name      None   None   None      Siemens AG
DeviceID         None   None   None          591664
VendorID         None   None   None              42
```

3.1.3. The Class `IOLinkMaster` (inherits from `MfMbCtrl`)

The Class `IOLinkMaster` provides an interface for communication with ET200eco PN. It implements methods for reading and writing (parameterizing) IO-Link devices, for reading process data from IO-Link devices, and for automatically detecting all devices connected to the master.

Here's a summary of the class's properties:

- Initialization: The same procedure applies here as for the ET200SpMf class. The Class `IOLinkMaster` allows initialization of communication with the ET200eco PN hardware via the specified host IP address, port number and the available ports for communication. In addition, the constructor `__Init__` the instance variable `self.iolink_devices` in the form of a DataFrame to store IO-Link device information in a structured way.
- Configuring IO-Link Devices: The "`liolink_device()`" enables the configuration of IO-Link devices via Modbus communication. It supports tasks such as reparameterization, diagnostics and execution of port functions. It returns the data read by the IO-Link device or written to the device in the form of an array of hexadecimal numbers.
- Automatic Detection of IO-Link Devices: The "`find_iolink_devices()`" finds and retrieves information about IO-Link devices connected to the IO-Link master (ET200eco PN). It updates the IO-Link Master's Device Information DataFrame (`self.iolink_devices`) with device IDs, manufacturer IDs, manufacturer names, and product names.
- Reading IO data: Use the "`read_io_data()`" to read IO data from the sensors. It reads the temperature from a temperature sensor connected to a port on the master.
- Helper Method: Removing Leading Zeros: The Helper Method "`remove_trailing_zeros()`" removes leading zeros from an array (0x000A => 0xA) and improves data processing and readability.

`IOLinkMaster` inherits from the class `MfMbCtrl`, which means that all methods of `MfMbCtrl` also in the class `IOLinkMaster` can be used.

Example usage

This example shows how you can use the `IOLinkMaster`class for communication with an ET200eco PN.

- Example of the creation of a client, the acyclic reading of IO-Link data from an IO-Link device using an IOL-Call, and the automatic identification of all devices connected to the master:

Code:

```
# Initialisierung eines ET200eco PN - Clients
mfMbClient_ECOPN = IOLinkMaster(_host_address="192.168.1.11", _modbus_port=502,
_iolink_master_ports=["Port 1","Port 2","Port 3", "Port 4", "Port 5", "Port 6",
"Port 7", "Port 8"])

# Auslesen von Direct Parameters eines IO-Link Devices
iolink_data = mfMbClient_ECOPN.iolink_device(_modbus_channel=0, _slot=1,
_sub_slot=1, _port=4, _cap=227, _readWrite=0, _index=0, _subindex=0, _length=20,
_writingRecord=[])
print("Direct Parameters:", iolink_data)

# IO-Link Devices suchen und anzeigen
mfMbClient_ECOPN.find_iolink_devices(_modbus_channel=0, _slot=1, _sub_slot=1)
print(mfMbClient_ECOPN.iolink_devices)
```

Debugging:

```
Direct Parameters:  ['0x804', '0xfe4a', '0x0', '0x0', '0x32', '0x3211', '0x1183', '0x1000', '0x2a09', '0x730']
                Port 1 Port 2 Port 3          Port 4 Port 5 Port 6 Port 7 Port 8
Identification
Product Name      None   None   None  3RS2800-*BA40   None   None   None   None
Vendor Name       None   None   None       Siemens AG   None   None   None   None
DeviceID          None   None   None           591664   None   None   None   None
VendorID          None   None   None               42   None   None   None   None
```

3.2. Integration into the user project

The Python Script `main.py` generates a user interface and populates it with hardware data that is retrieved using the libraries mentioned above. The user interface (UI) was created with "Bokeh" (see \5\), a Python library designed for creating interactive visualizations in the web browser. The UI is divided into two tabs: ET200SP MF and ET200ecoPN.

The comprehensive diagnostic information and overview of hardware components, including specific details such as firmware version and hardware revision, are presented in a table. In addition, graphical elements, especially diagrams, are used for the visual representation of data such as temperature profiles, energy consumption and current peaks in order to convey this information effectively.

Updating the charts with new data requires that data to be retrieved or streamed. Bokeh's periodic recall functions are used for this purpose. They automatically update data at regular intervals. A bokeh server is necessary to effectively use these callback functions because the web application is not static HTML. Setting up the bokeh server ensures that the charts remain dynamic and responsive while continuously updating them with the latest data available. This allows for an efficient and timely presentation of the changing information in the user interface.

NOTE Note that the information provided is merely examples of how to use the above libraries.

Bokeh Documentation
You can find the complete documentation for Bokeh at https://docs.bokeh.org/en/latest/ (external link)

- The UI - ET 200SP MF tab with diagnostic data and I&M data (Part 1)

ET200SP MF | ET200ecoPN

Diagnose Update

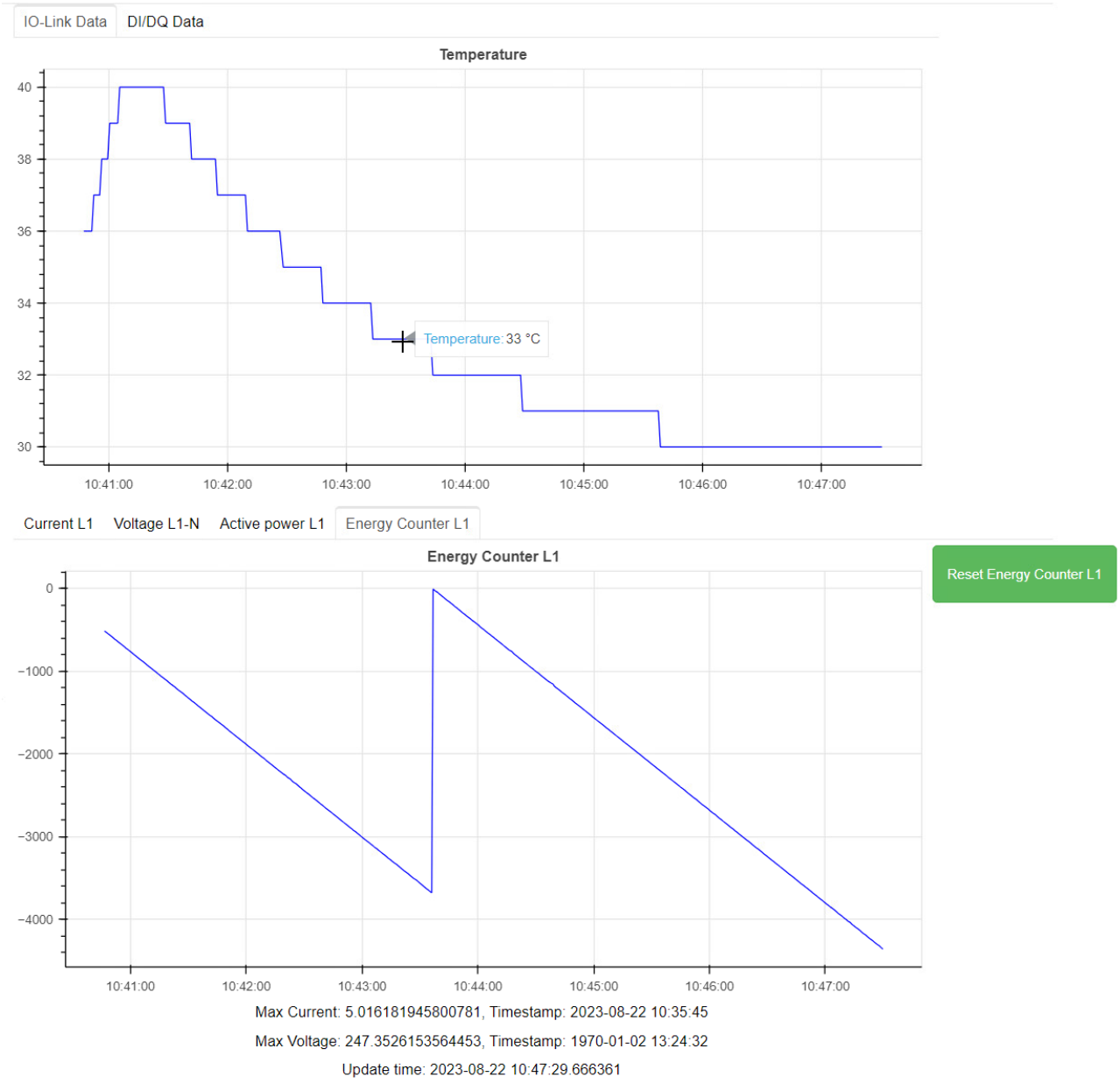
Slot	Submodule	Channel number	Channel type	Error code	Error message
1	CM 4xIO-Link ST	4	Reserved	0x7	Oberer Grenzwert übersch
1	CM 4xIO-Link ST	4	Reserved	0x6	Drahtbruch

Identification and Maintenance (I&M)

MF Station | CM 4xIO-Link

I&M - Information	IM155-6MF HF	CM 4xIO-Link ST	DQ 8x24VDC/0.5A HF	DI 8x24VDC HF	AI Energy Meter ST
Article Number	6ES7 155-6MU00-0CN0	6ES7 137-6BD00-0BA0	6ES7 132-6BF00-0CA0	6ES7 131-6BF00-0CA0	6ES7 134-6PA21-0CU0
Serial Number	S C-P3MR22772022	S C-D2VA99582013	S C-JDNH47282017	S C-JDLV90782017	S C-P3QD65812022
HW Revision	3	1	5	6	2
FW Version	V 5.2.2	V 2.2.2	V 2.0.2	V 2.0.1	V 8.0.0

- The UI - ET 200SP MF tab with diagrams filled with process data (Part 2)



3.3. Operation

A Python interpreter is required to execute the code in this use case. The Python script uses different libraries for functionality and user interface.

After all the libraries have been installed, you can start a bokeh server running the web application. To do this, use the following command in the terminal:

```
$ bokeh serve --show main.py
```

Use the "Update" button to update the diagnostic table. Note that diagnostics are not automatically updated periodically, only at the user's request.

The "Reset Energy Counter L1" button on resets the energy meter. (In the figure above, the energy meter was reset once between 10:43 and 10:44.)

This application example shows how you can retrieve data from the field level in parallel to the automation field without influencing or changing the PROFINET automation solution. This approach avoids changes to the control program and thus an unnecessary load on the PLC. This maintains operational efficiency without the need for hardware upgrades.

4. Appendix

4.1. Service and support

SiePortal

The integrated platform for product selection, purchasing and support - and connection of Industry Mall and Online support. The SiePortal home page replaces the previous home pages of the Industry Mall and the Online Support Portal (SIOS) and combines them.

- Products & Services
  In Products & Services, you can find all our offerings as previously available in Mall Catalog.
- Support
  In Support, you can find all information helpful for resolving technical issues with our products.
- mySieportal
  mySiePortal collects all your personal data and processes, from your account to current orders, service requests and more. You can only see the full range of functions here after you have logged in.

You can access SiePortal via this address: sieportal.siemens.com

Technical Support

The Technical Support of Siemens Industry provides you fast and competent support regarding all technical queries with numerous tailor-made offers – ranging from basic support to individual support contracts.
Please send queries to Technical Support via Web form: support.industry.siemens.com/cs/my/src

SITRAIN – Digital Industry Academy

We support you with our globally available training courses for industry with practical experience, innovative learning methods and a concept that's tailored to the customer's specific needs.
For more information on our offered trainings and courses, as well as their locations and dates, refer to our web page: siemens.com/sitrain

Industry Online Support app

You will receive optimum support wherever you are with the "Industry Online Support" app. The app is available for iOS and Android:

4.2. Links and Literature

No.	Topic
\1\	Siemens Industry Online Support https://support.industry.siemens.com
\2\	Link to the contribution page of the application example https://support.industry.siemens.com/cs/ww/en/view/109987554
\3\	SIMATIC MultiFieldbus - Function Manual https://support.industry.siemens.com/cs/document/109773209
\4\	Modbus/TCP communication with MultiFieldbus and Raspberry Pi. https://support.industry.siemens.com/cs/attachments/109808268/109808268_ModbusTCP_MF_DOC_V10_en.pdf
\5\	Bokeh documentation https://docs.bokeh.org/en/latest/
\6\	SiePortal application example 109754890, ET 200SP timestamp function in conjunction with the AI Energy Meter 480V AC HF
\7\	SiePortal Download 82981502, Library for IO-Link (LIOLink)
\8\	SiePortal Manual 19289930, From PROFIBUS DP to PROFINET IO

4.3. Change documentation

Version	Date	Alteration
V1.0	04/2025	First edition

